

Know Less, Read More:

Toward a Minimal Base Curriculum for Small In-Context Learners

Davitotty1

Independent Researcher

github.com/davitotty

Abstract

Small language models are usually judged by how much they know. We ask how little a model needs to know if the rest can be read from its context. We frame this as the search for a *base curriculum*: the minimal set of skills that must be in the weights so that task-specific knowledge can be supplied as instructional material at inference time. We propose a six-layer candidate, a criterion for what belongs in the weights, and a ladder of generalisation that separates memorising a textbook from reading one. We introduce TinyTextbook, a synthetic benchmark in which each episode is a small textbook of freshly sampled rules followed by exercises, and report a pilot study with transformers of about 113 thousand parameters trained on a CPU (38 runs). Models trained on 16 or fewer distinct textbooks answer perfectly on those and at or near chance on new ones; with enough distinct textbooks they read new ones. A skill left out of training stays at chance, and without the simplest skill the models did not acquire composition either. Separately trained skills did not combine: no run exceeded 26.2% on exercises mixing two of them (guessing: 26.7%). Applying the same two layers twice extended the chain of rules a model could follow without adding parameters, but at equal depth looped and unshared models showed no consistent difference. Outcomes vary between seeds and are reported as counts. The pilot is small: one to four seeds, one synthetic domain, and a fixed page layout in which values are found by position; retrieval by content was not learned within our budget. We give a protocol with falsifiable hypotheses for 10–100 million parameters and release the code.

1 Introduction

A language model with a hundred million parameters has little room for an encyclopedia. What it might hold instead is the ability to *use* one. A student who opens a mathematics textbook does not need to have memorised its theorems; she needs to be able to read a definition, follow a worked example, apply a stated rule to a new case, chain two rules together, and notice when an answer does not check out. The theorems stay in the book. This paper asks what the corresponding division of labour looks like for a small transformer: *what is the least that must be stored in the weights so that everything else can be supplied in the context?*

That such a division is possible at all is the lesson of in-context learning (ICL). Large language models adapt to new tasks from a prompt without any weight update (Brown et al., 2020), and transformers trained from scratch on synthetic data learn to fit whole function classes from in-context examples (Garg et al., 2022), in some cases generalising to datasets that were never part of meta-training (Hollmann et al., 2023; Kirsch et al., 2022). The ability is itself learned: the weights do not encode the answer, they encode a procedure for extracting the answer from what is in front of the model (Akyürek et al., 2023; Bai et al., 2023; von Oswald et al., 2023).

Two features of this literature leave our question open. First, the material that is placed in context is almost always a list of (x, y) exemplars drawn from a single function class. A textbook is a different object: it states rules explicitly, mixes rules with examples, introduces its own notation, and expects the reader to

combine several of its statements to answer one exercise. Second, the question most often asked about pretraining is quantitative: *how many* distinct tasks must be seen before in-context learning emerges (Goddard et al., 2025; He et al., 2024; Nguyen and Reddy, 2025; Raventós et al., 2023). The complementary question is qualitative: *which* abilities form the set that has to be in the weights, and which can be left out without loss. We call that set the *base curriculum*.

Answering the qualitative question matters most for small models. When capacity is scarce, every fact stored in the weights displaces something else, and a fact that could have been read from the context is the cheapest thing to displace. If the base curriculum is small and mostly procedural, then a small model that is trained on it and paired with good material could be far more useful than its parameter count suggests. If instead the base curriculum turns out to be large, or impossible to separate from domain knowledge, that would be an equally useful thing to know.

Contributions.

1. **A formulation** of the problem (Section 3): open-book learners, a simple admission criterion for what belongs in the weights, and a four-level ladder of generalisation that separates memorising a textbook from reading one.
2. **A concrete hypothesis** (Section 4): a six-layer base curriculum, from parsing notation up to verifying an answer, together with an explicit list of what it excludes.
3. **TinyTextbook** (Section 5): a synthetic benchmark in which every episode is a small textbook followed by exercises, and in which the rules are resampled for every episode so that memorising them is useless. The generator, the model and the training loop fit in one file of about 300 lines, released with this paper.
4. **A pilot study** (Section 6) with transformers of roughly 113 thousand parameters trained on a CPU. Models trained on few distinct textbooks answer perfectly on those and near chance on new ones, while models trained on many read new textbooks; a skill left out of training stays at chance, and leaving out the simplest skill also blocks composition; separately trained skills do not combine without training; and applying the same layers a second time extends the chain of rules a model can follow at no cost in parameters. A fourth experiment shows that a symmetry in the training data can keep the basic reading skill from being learned within the budget, and we report what failed.
5. **A protocol** (Section 7) with falsifiable hypotheses for repeating the study at 10–100 million parameters, where the question actually has practical consequences.

Scope. The pilot is deliberately tiny, uses one synthetic domain, and covers four of the six proposed layers. Its role is to show that the question can be posed precisely and measured cheaply, and to report what the first measurements say. It supports no claim about natural language or about models of practical size; Section 8 lists what would have to be true for the picture to carry over.

2 Related Work

In-context learning as a learned algorithm. Garg et al. (2022) showed that transformers trained from scratch can learn linear functions, sparse linear functions, decision trees and small neural networks from in-context examples. Subsequent work characterised the algorithm being implemented, relating it to gradient descent and ridge regression (Akyürek et al., 2023; von Oswald et al., 2023), to in-context algorithm selection (Bai et al., 2023), and to implicit Bayesian inference (Xie et al., 2022). At the circuit level, Olsson et al. (2022) identified induction heads as a mechanism for copying from context. Prior-data fitted networks apply

the same idea to tabular prediction, meta-training on synthetic datasets only (Hollmann et al., 2023; Müller et al., 2022).

When does in-context learning emerge? Chan et al. (2022) traced the emergence of ICL to distributional properties of the training data and documented a trade-off with in-weights learning; Singh et al. (2023) found that ICL can emerge and then fade as training continues. Raventós et al. (2023) identified a task-diversity threshold: below it a transformer behaves like a Bayesian estimator whose prior is the finite set of pretraining tasks, and above it the model solves tasks it has never seen. Kirsch et al. (2022) described transitions between algorithms that generalise, algorithms that memorise, and failure to meta-train, and argued that the accessible state size, more than the parameter count, limits a meta-trained in-context learner. Related transitions have been reported for modular arithmetic (He et al., 2024) and analysed in terms of the pretraining task distribution (Goddard et al., 2025; Letey et al., 2025) and of learning dynamics (Nguyen and Reddy, 2025; Reddy, 2023). These studies ask how much diversity is needed; we ask which skills the diverse tasks must exercise.

Meta-training language models to learn in context. MetaICL (Min et al., 2022) and in-context tuning (Chen et al., 2022) fine-tune language models on collections of tasks so that they learn new tasks from prompts. Lake and Baroni (2023) meta-train a transformer on a stream of episodes with changing grammars and obtain systematic compositional generalisation, addressing the failure documented by Lake and Baroni (2018). Our setting follows the same recipe of episodes with resampled rules, applied to instructional material and organised around the question of which skills are necessary.

Learning from material placed in the context. The closest formal treatment is that of Zhu et al. (2025b), who study gradient-based training in which helpful material is present in the context and show, in a simplified setting, an exponential gain in sample efficiency when the model is capable of ICL. Russin et al. (2025) show that in-context and in-weight learning respond differently to the ordering of examples, reproducing curriculum effects known from human learning. At the system level, retrieval-augmented generation supplies documents to the model at inference time (Lewis et al., 2020); our question is what the model reading those documents must already be able to do, and Liu et al. (2023) show that the reading itself is not robust: performance drops when the relevant information sits in the middle of a long context.

Small models and curricula. Restricting and curating the training distribution lets very small models acquire abilities usually associated with larger ones, for instance coherent text (Eldan and Li, 2023), code (Gunasekar et al., 2023) and arithmetic (Lee et al., 2023). In that line of work the curated data are absorbed into the weights; here the “textbook” stays in the context. Ordering training from easy to hard is an old idea (Bengio et al., 2009; Elman, 1993) that has recently been applied to reasoning in small language models (Fu, 2025).

Recurrent depth. Composing several rules read from the context is a serial computation, and the depth of a transformer bounds how much of it fits into one forward pass. Re-applying the same layers is one way to buy depth without parameters. The idea goes back to Universal Transformers (Dehghani et al., 2019) and adaptive computation time (Graves, 2016). Looped transformers can emulate programmable computers (Giannou et al., 2023), learn in-context learning algorithms with a fraction of the parameters of an unshared stack (Yang et al., 2024), and approach the performance of much deeper unshared models on reasoning tasks (Saunshi et al., 2025). The approach has been scaled to language-model pretraining (Bae et al., 2025; Geiping et al., 2025; Zhu et al., 2025a), and recursion is central to recent small models for abstract puzzles (Jolicoeur-Martineau, 2025; Wang et al., 2025). Computation carried in hidden states can replace computation written out as a chain

of thought (Hao et al., 2025; Nye et al., 2021; Wei et al., 2022), which makes it harder to inspect (Korbak et al., 2025); we return to this in Section 8.

3 Formulation

3.1 Open-book learners

A *task* is a rule set R together with a semantics that assigns to every query q an answer $a = \llbracket q \rrbracket_R$. A *textbook* for R is a token sequence m that specifies R , fully or in part; an *exercise* is a pair (q, a) . A *closed-book learner* is a map $q \mapsto \hat{a}$ and can answer correctly only if R is encoded in its parameters. An *open-book learner* is a map $(m, q) \mapsto \hat{a}$. An ideal open-book learner implements an interpreter: it recovers R from m and evaluates q under it, for every R in some class $\mathcal{R}$ and every faithful rendering m of R .

This separates two kinds of knowledge. *Declarative* knowledge is the content of R : it differs from task to task. *Procedural* knowledge is whatever is needed to implement the interpreter: it is shared by all tasks in $\mathcal{R}$. The working hypothesis of this paper is that a small model should spend its parameters on the second kind and receive the first kind through m .

3.2 What belongs in the weights

Consider one item of knowledge z : a fact, a definition, or a procedure. We propose that z belongs in the weights if it passes either of two tests.

Bootstrap. z is needed to interpret the material itself. The meaning of the notation in which definitions are written cannot be supplied by a definition written in that notation.

Amortisation. Stating z in the context every time it is needed costs more than storing it once. If z is used in a fraction ρ_z of N deployment queries and takes c_z tokens to state, at a cost κ per token, while storing it takes b_z bits of capacity priced at λ per bit, then z belongs in the weights when

$$\rho_z N c_z \kappa > \lambda b_z. \quad (1)$$

Inequality (1) is a framing device and we do not attempt to measure its terms. Its use is to make two qualitative predictions. A procedure that almost every query exercises (large ρ_z) is worth storing even if it is expensive. A fact that few queries need (small ρ_z) is not, and the smaller the model (the larger λ), the more facts fall on that side of the line.

3.3 A ladder of generalisation

High accuracy on exercises does not by itself show that a model reads. We distinguish four levels, each of which removes one way of succeeding without reading.

G0: seen textbook, new exercise. The rules were present in training. A closed-book learner can succeed.

G1: new textbook. The rules are freshly sampled from the families used in training. Success requires reading the material.

G2: new combination of skills. The exercise combines trained skills in a way never seen in training, for example a longer chain of rules, or a chain that mixes a stated rule with an induced one.

G3: new family. The material defines a kind of rule that never occurred in training.

	Layer	The model can...	Test passed
C0	Parsing	segment the material into definitions, examples and exercises, and bind names to what they denote	Bootstrap
C1	Primitives	copy a token, retrieve a value by its key, substitute, compare, count, do small-number arithmetic	Amortisation
C2	Rule following	apply a rule that the material states explicitly	Amortisation
C3	Rule induction	infer a rule that the material only illustrates with examples	Amortisation
C4	Composition	chain several rules, feeding the output of one into the next	Amortisation
C5	Verification	check an answer against the rule, and abstain when the material does not determine the answer	Amortisation

Table 1: The proposed base curriculum. Everything on this list is procedural. The last column gives the test of Section 3.2 under which the layer is admitted.

A set of skills is a *sufficient* base curriculum at level Gk for a domain if a model trained only on it reaches high accuracy at Gk , and *minimal* if removing any one skill breaks this. The pilot in Section 6 measures $G0$, $G1$ and $G2$; $G3$ is left to the protocol of Section 7.

4 The Base-Curriculum Hypothesis

Table 1 states the hypothesis. It is a proposal to be tested, and the tests are leave-one-out ablations: train without one layer, then measure what breaks.

What is excluded. Theorems, formulas, constants, named algorithms, and domain procedures such as “how to solve a quadratic equation” are left out. Each is a rule that a textbook can state, so each can travel in the context and be handled by C2 and C4.

Where the line is uncertain. Three boundary cases deserve mention, because they are where we expect the hypothesis to be revised. First, small-number arithmetic is placed in C1 on the grounds that it is used by almost every mathematical rule, but it is declarative in nature and could in principle be supplied as a table; the pilot’s induction skill depends on it. Second, the layers need not be separable in training: an exercise that requires composition necessarily exercises rule following, so “training without C2” can only mean removing exercises that consist of rule following alone. Third, the choice of what counts as one rule family (C3) is a modelling decision that determines how hard $G3$ is.

5 TinyTextbook

TinyTextbook is the smallest setting we could design in which the layers C1–C4 and the levels $G0$ – $G2$ are all defined.

Episodes. Let Σ be an alphabet of $K = 5$ symbols. A textbook contains three functions $f_1, f_2, f_3 : \Sigma \rightarrow \Sigma$ whose values are fully tabulated, and a fourth function g of which only two input–output examples are

Material (19 tokens after the start token)		
row of f_1 :	c e d a b	<i>i.e. $f_1(a)=c, f_1(b)=e, f_1(c)=d, \dots$</i>
row of f_2 :	e b c a d	
row of f_3 :	a c d e b	
examples of g :	c e b d	<i>i.e. $g(c)=e, g(b)=d$</i>
Exercises		
FOLLOW	Q F2 a = <u>e</u>	<i>read one entry</i>
COMPOSE	Q F1 F2 d = <u>c</u>	<i>$f_2(d)=a$, then $f_1(a)=c$</i>
INDUCE	Q G a = <u>c</u>	<i>g shifts by two; a is not among the examples</i>
MIXED	Q G F3 a = <u>c</u>	<i>$f_3(a)=a$, then $g(a)=c$; never seen in training</i>

Figure 1: One TinyTextbook episode in the aligned layout. The model is trained to predict only the underlined answers. Every function is resampled for each new episode.

shown. The hidden rule behind g is a cyclic shift, $g(x) = x + s \bmod K$ for a shift $s \in \{1, \dots, K-1\}$ under a fixed ordering of Σ , so the examples determine it completely, but only for a reader who knows that ordering and can add. The material is followed by exercises written in prefix notation: $Q \ h_d \ \dots \ h_1 \ x =$ asks for $h_d(\dots h_1(x))$, and the next token is the answer. Figure 1 shows an example. All four functions are resampled for every episode.

Skills. Four kinds of exercise map onto the curriculum. FOLLOW asks for one tabulated value and exercises C1 and C2. COMPOSE chains $d \geq 2$ tabulated functions and exercises C4. INDUCE queries g on an input that is not among its examples and exercises C3. MIXED places g inside a chain of tabulated functions, with the constraint that the input reaching g is not one of its examples; it is never used in training and serves as a G2 test, as do COMPOSE exercises that are deeper than any seen in training.

Two layouts. In the *aligned* layout, used for all results below, the material is a table with a fixed geometry: row j lists $f_j(a), \dots, f_j(e)$ in order, and the function in row j always carries the name token Fj . A value is found by its address, while what is stored at each address changes with every episode. The page format is thus in the weights (C0) and the content is in the context. In the *shuffled* layout each fact is a free-standing triple `name key value`, the triples appear in random order, and the names are redrawn from a pool of eight for every episode, so a value has to be found by matching content. Section 6.5 reports that the shuffled layout was not learned within our budget.

Training on maps, testing on permutations. In training episodes the tabulated functions are arbitrary maps $\Sigma \rightarrow \Sigma$; in test episodes they are permutations. With permutations every symbol appears exactly once in every row, so the answer to a single look-up is uniform over Σ unless the entry is read correctly, and the chance level for FOLLOW is exactly $1/K = 20\%$. For chains the level is slightly higher, because a function can occur twice in a chain and $f(f(x)) = x$ with probability $2/K$: a guesser that sees the exercise but not the tables reaches 26.7% at length 2, 22.2% at length 3 and between 20.9% and 23.7% at lengths 4 to 6. We call these the *guessing levels*; Appendix A gives their derivation. The reason for training on arbitrary maps is empirical and is the subject of experiment E4 (Section 6.4).

Measurement. Training episodes contain eight exercises and the loss is the cross-entropy on the eight answer tokens. Test episodes contain a single exercise placed directly after the material, so that no previously worked exercise can be used as a hint.

Distinct textbooks in training (M)	G0: seen textbooks		G1: new textbooks	
	FOLLOW	COMPOSE-2	FOLLOW	COMPOSE-2
1	100.0	100.0	20.7	21.3
4	100.0	100.0	22.8	21.2
16	100.0	100.0	31.4	21.1
64	100.0 2/2	99.8 2/2	90.5 1/2	51.3 0/2
∞	100.0 2/2	100.0 2/2	100.0 2/2	100.0 2/2

Table 2: E1: accuracy (%) after training on M distinct textbooks. One seed for $M \leq 16$; for $M = 64$ and $M = \infty$ the mean of two seeds, followed by the number of seeds that ended at 95% or above. On new textbooks the guessing level is 20% for FOLLOW and 26.7% for COMPOSE-2. Seen textbooks tabulate arbitrary maps, as in training; new textbooks tabulate permutations.

6 Pilot Study

The pilot asks three questions of the smallest models that can answer them. Does a model trained on textbooks read them or memorise them (E1)? Which exercise types must be in the training set for which abilities to appear (E2)? Does re-applying the same layers extend the chain of rules a model can follow (E3)? A fourth experiment (E4) isolates an effect of the training distribution that we ran into while designing the benchmark.

Setup. All models are decoder-only transformers of width 64 with four attention heads. The default model applies a block of two layers twice, for four layer passes and 113,297 parameters. Training takes 5,000 steps of 32 episodes (8,000 steps in E3) on one CPU core, which is between 6 and 23 minutes per run. Accuracy is measured on 4,000 held-out episodes per test, giving a standard error below 0.8 percentage points. Appendix A gives the remaining details, and Appendix B lists every run. The pilot comprises 38 runs and 5.4 CPU-hours. The number of seeds per condition is small and is stated with each result; we therefore describe only effects that are large compared with the gap between the guessing level and ceiling.

6.1 E1: Memorising a textbook versus reading one

Every training episode draws its textbook from a fixed pool of M textbooks, or from an unlimited supply ($M = \infty$). The exercises are sampled afresh each time, half of them FOLLOW and half COMPOSE at depth 2, and the number of training episodes is the same for every M . We then test on textbooks from the pool (G0) and on new textbooks (G1).

Table 2 and Figure 2a show the outcome. Every model answers exercises about its training textbooks almost perfectly. For $M \leq 16$ that is all it can do: on a new textbook, FOLLOW accuracy is 20.7%, 22.8% and 31.4% for $M = 1, 4, 16$. These models have stored their textbooks in the weights and do not consult the page in front of them. With $M = 64$ the same architecture, trained for the same number of steps, begins to use textbooks it has never seen: over two seeds it follows their rules at 100.0 and 81.0% and composes them at 74.2 and 28.4%. With an unlimited supply, both seeds reach at least 100.0% on FOLLOW and 100.0% on COMPOSE.

The switch from memorising to reading as the number of distinct textbooks grows is the instructional-material counterpart of the task-diversity threshold reported for in-context regression (Raventós et al., 2023) and for modular arithmetic (He et al., 2024). Four points are worth drawing out.

Accuracy on seen material is uninformative. The G0 columns are the same, to within half a point, for models that read and models that do not. Any evaluation in which the rules were available during training, in whatever form, cannot distinguish the two.

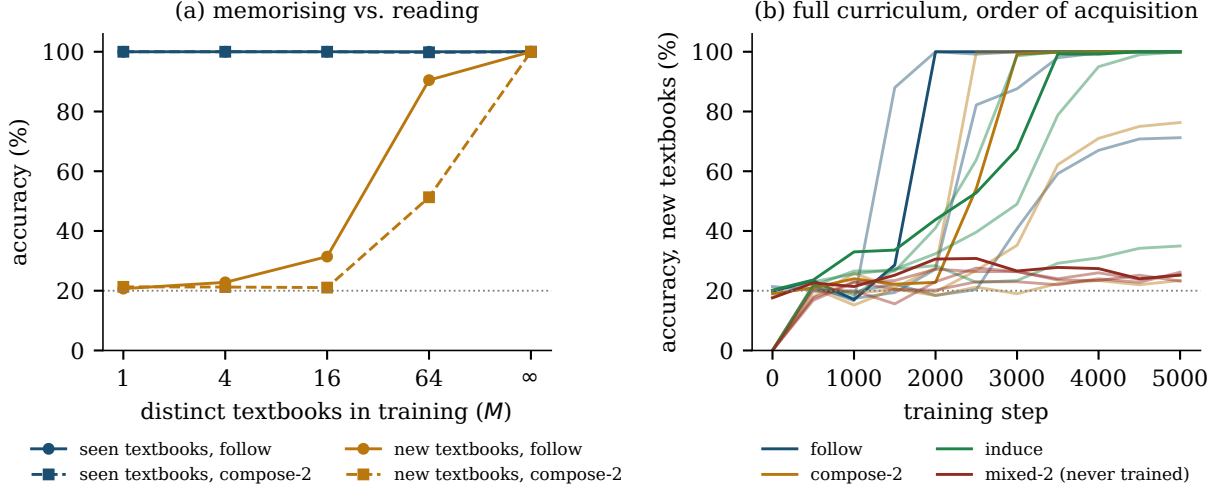


Figure 2: (a) E1 (means over seeds). All models answer almost perfectly on their training textbooks; only those trained on enough distinct textbooks can use a new one. (b) E2, full curriculum, accuracy on new textbooks during training for each seed (later seeds are drawn lighter). The skills are acquired one after another in abrupt steps, and the never-trained combination MIXED stays at the guessing level. Dotted line: 20%.

Variety made the difference, not volume. Every model saw the same 160,000 training episodes. What changed is how many different textbooks those episodes were about. This is the first half of hypothesis H3 in Section 7.

The harder skill asks for more variety. At $M = 64$, FOLLOW transfers to new textbooks largely or completely (100.0 and 81.0%) while COMPOSE lags behind (74.2 and 28.4%). With one or two seeds per condition we do not read more into the location of the threshold than that reading begins somewhere between 16 and 64 textbooks for the easier skill and is not complete at 64 for the harder one.

Knowledge in the weights did not help with a new page. The models with $M \leq 16$ know their textbooks perfectly and are at or near chance on a new one. This bears on hypothesis H2 only indirectly: the direct test, adding facts to the weights of a model that already reads, is left to the full-scale study.

6.2 E2: Which skills have to be trained

We train four models on new textbooks only ($M = \infty$): one on all three exercise types in equal proportion, and three with one type left out and the remaining two in equal proportion. COMPOSE is trained at depth 2 only. All four are tested on the three trained types and on two G2 tests: MIXED at depth 2, which combines COMPOSE with INDUCE, and COMPOSE at depth 3.

Table 3 gives the result, and the first thing it shows is how uneven learning is at this budget. Skills are acquired in abrupt steps (Figure 2b), so a run ends either near ceiling or near chance, and whether a given step arrives before training stops varies with the seed. Even FOLLOW, the simplest skill, was solved in only 9 of the 12 runs whose training set contained it. Alongside each mean we therefore report how many seeds ended at 95% or above, and we read the table by comparing those counts.

A skill left out of training was never acquired. Without INDUCE exercises, INDUCE accuracy is at or below its guessing level in every seed (20.7, 20.6, 20.6 and 19.5%); without COMPOSE exercises, so is COMPOSE (17.4, 19.3, 20.6 and 20.8%). Following and composing do not yield induction, and following and

Training set	G1: exercise types in the curriculum			G2: never trained	
	FOLLOW	COMPOSE-2	INDUCE	MIXED-2	COMPOSE-3
Full curriculum	92.8 3/4	74.9 2/4	83.7 3/4	25.0 0/4	20.5 0/4
without FOLLOW	19.7 0/4	25.3 0/4	81.4 2/4	22.0 0/4	21.1 0/4
without COMPOSE	73.8 2/4	19.5 0/4	99.8 4/4	20.0 0/4	20.5 0/4
without INDUCE	100.0 4/4	100.0 4/4	20.4 0/4	17.0 0/4	20.4 0/4

Table 3: E2: accuracy (%) on new textbooks after training with one exercise type left out. Each cell gives the mean over seeds, followed by the number of seeds that ended at 95% or above out of the number run. Guessing levels: 20% for FOLLOW, 25% for INDUCE, 26.7% for COMPOSE-2, 22.2% for COMPOSE-3, and 26.7% on MIXED-2 for a model that has induced g but cannot combine it with a table look-up (see text).

inducing do not yield composition. Within TinyTextbook each skill is therefore necessary for itself, which is the minimality condition of Section 3.3 at level G1.

Leaving out the simplest skill removed more than that skill. The models trained on COMPOSE and INDUCE, but never on a single look-up, did not learn to look up (19.1, 19.7, 19.4 and 20.8% on FOLLOW) and did not learn to compose either (26.4, 25.9, 25.0 and 23.8%, which is the guessing level for two-step chains). COMPOSE was solved in 0 of 4 runs without FOLLOW in the training set, against 6 of 8 runs with it (the full curriculum and the one without INDUCE; one-sided Fisher exact test, $p = 0.030$). A two-step exercise contains two look-ups, so in principle it supervises everything a one-step exercise does. In practice the one-step exercise acted as a stepping stone without which the two-step skill was not found within the budget. This also answers, for this benchmark, the worry raised in Section 4 that rule following cannot be removed separately from composition: it can, and removing it matters.

The full curriculum was not learned in every seed. With all three exercise types in training, 3 of 4 seeds solved FOLLOW, 2 solved COMPOSE and 3 solved INDUCE; 2 solved all three. Across all conditions, the runs that fell short either made their first transition late, when the learning rate had already decayed too far for the remaining skills to follow, or never made it. Nothing in our data explains which seeds these are. The practical consequence is that a leave-one-out comparison at a fixed budget must be read through counts over seeds, as above, and Section 7 lists the corrections we would make to the design.

Trained skills did not combine by themselves. MIXED exercises ask for a tabulated function and the induced function in one chain. The models trained on the full curriculum score 25.0% on them (per seed: 25.2, 26.2, 23.2 and 25.3). This is no better than guessing: when g is the outer function, its input cannot be one of the two example keys, so the answer is one of three symbols that the examples determine, and when g is the inner function the answer is uniform; a model that has induced g and otherwise guesses therefore reaches $(1/3 + 1/5)/2 = 26.7\%$. None of the 16 runs in this experiment solved MIXED; the best reached 26.2%. Having each skill in the weights was not enough to obtain their combination, which contradicts hypothesis H5 at this scale. COMPOSE at depth 3 is likewise at its guessing level in every run, as expected from four layer passes trained on two-step chains; E3 examines depth directly.

6.3 E3: Recurrent depth

All models in this experiment are trained on COMPOSE exercises whose chain length is 1, 2, 3 or 4 with equal probability (length 1 is FOLLOW), and are tested at lengths 1 to 6. We compare a block of two layers applied once, twice and three times with unshared stacks of four and six layers. A looped model with r loops and an

Model	Params	Layer		G1: chain lengths in training				G2: longer	
		passes	Seeds	1	2	3	4	5	6
2 layers $\times$ 1 loop	113k	2	1	100.0	24.9	24.4	22.9	21.4	22.1
2 layers $\times$ 2 loops	113k	4	3	100.0 _{3/3}	100.0 _{3/3}	71.9 _{1/3}	23.4 _{0/3}	22.2 _{0/3}	21.8 _{0/3}
2 layers $\times$ 3 loops	113k	6	1	100.0	100.0	46.7	23.2	23.0	22.3
4 layers, unshared	213k	4	3	100.0 _{3/3}	89.3 _{1/3}	55.0 _{1/3}	31.3 _{0/3}	21.6 _{0/3}	21.8 _{0/3}
6 layers, unshared	313k	6	1	100.0	95.2	65.1	25.1	22.1	22.8

Table 4: E3: accuracy (%) on new textbooks by chain length. Where a model has several seeds, the mean is followed by the number of seeds that ended at 95% or above out of the number run. Guessing levels for lengths 1 to 6: 20.0, 26.7, 22.2, 23.7, 20.9 and 22.4%.

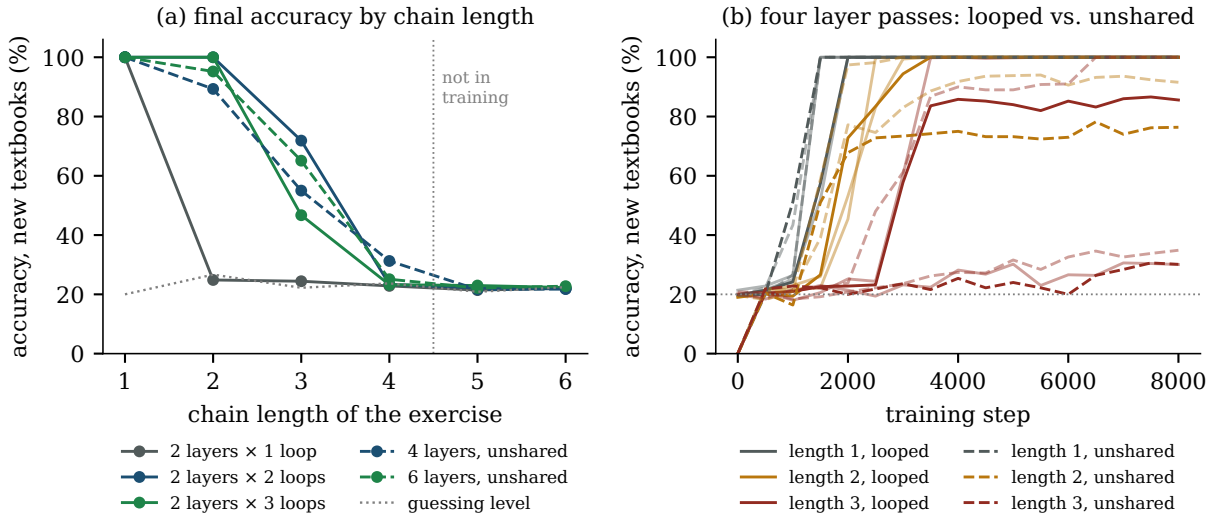


Figure 3: E3. (a) Final accuracy by chain length, averaged over seeds; solid lines are looped models, dashed lines unshared ones, colour marks the number of layer passes, and the dotted grey curve is the guessing level. (b) Accuracy during training for the two models with four layer passes, one curve per seed (later seeds lighter). The chain lengths are acquired one after another, at times that differ widely between seeds. Dotted horizontal line: 20%.

unshared model with $2r$ layers perform the same number of layer passes, and the looped one has $1/r$ of the transformer-block parameters.

Loops supplied depth without adding parameters. With two layer passes the model learned single look-ups and nothing longer (24.9% at length 2, below the guessing level; one seed). Applying the same two layers a second time, with not one parameter added, was enough for length 2 to be solved in all 3 seeds and for length 3 to reach 71.9% on average (per seed: 85.6, 100.0 and 30.0). This is hypothesis H4 as stated in Section 7: at a fixed parameter count, a second loop raised the chain length the model could follow.

At equal depth, looped and unshared models showed no consistent difference. The unshared four-layer model has the same number of layer passes as the two-loop model and 213,265 parameters instead of 113,297. Over 3 seeds each, accuracy at length 2 was 100.0, 100.0 and 100.0% for the looped model and 76.4, 100.0 and 91.5% for the unshared one; at length 3 it was 85.6, 100.0 and 30.0% and 30.1, 100.0 and 34.9%; and at

Tables in training	Seeds	FOLLOW on new textbooks with		First checkpoint at $\geq 95\%$ (per seed)
		permutations	arbitrary maps	
Arbitrary maps	3	100.0 3/3	100.0 3/3	1,500, 2,000, 2,500
Permutations	3	38.7 0/3	31.6 0/3	never, never, never

Table 5: E4: FOLLOW accuracy (%) after training on FOLLOW only, with training tables that are arbitrary maps or permutations. Mean over seeds, followed by the number of seeds that ended at 95% or above out of the number run. The last column gives, per seed, the first of the checkpoints (taken every 500 steps) at which accuracy on new permutation textbooks was at least 95%. The guessing level is 20% on permutation textbooks; on textbooks of arbitrary maps a model can exceed it by guessing a frequent symbol.

length 4, 22.0, 23.0 and 25.1% and 22.7, 48.1 and 23.1%. The looped model was ahead at length 2 in two seeds and at length 3 in one; the unshared model was ahead in another seed, the only run in the experiment to rise clearly above the guessing level at length 4. The spread between seeds is as large as any difference between the architectures (Figure 3b), and with three seeds we do not claim a difference in either direction. What can be said is the weaker statement that sharing weights across depth cost nothing measurable here while halving the number of block parameters. Earlier work reports that weight sharing favours iterative computation (Saunshi et al., 2025; Yang et al., 2024); our data neither confirm nor contradict this at equal depth.

A third loop made no visible difference. The three-loop model learned length 2 completely and reached 46.7% at length 3, which is inside the range spanned by the two-loop seeds (one seed). In an exploratory run with a longer schedule of 10,000 steps the same model reached 95% at length 3 by step 6,500. We therefore have no evidence that a third loop helps and none that it hurts; the question needs more seeds and a longer budget than we had.

No model went beyond the lengths it was trained on. Accuracy at lengths 5 and 6 is at the guessing level for every model and every seed, including the models with six layer passes. Like the MIXED test of E2, this is a failure at level G2.

6.4 E4: A symmetric training distribution holds the reading skill back

Our first training sets tabulated permutations, as the test sets do, and in exploratory runs the models did not learn to look a value up (Appendix C). E4 examines this in isolation. Two groups of models are trained on FOLLOW exercises only, with identical settings except for one: in one group the tabulated functions of the training textbooks are arbitrary maps, in the other they are permutations. Both groups are tested on new textbooks of both kinds.

Table 5 shows the result. With arbitrary maps in training, all 3 seeds made an abrupt transition from chance to ceiling (first at 95% or above at steps 1,500, 2,000, 2,500) and ended at 100% on both kinds of new textbook. With permutations in training, no seed made that transition within 5,000 steps: accuracy crept above chance during the second half of training and ended at 30.3, 52.5 and 33.3% on new permutation textbooks. The comparison is tilted against the first group, which is tested on tables from a different distribution than the one it was trained on, while the second group is tested on its own training distribution; the group with the distribution shift is the one that learned.

Our explanation, which we have not tested directly, is the following. When every row is a permutation, the multiset of symbols in a row is the same in every episode, so an attention pattern that spreads over a row, or over the whole table, carries no information about the answer; only attention concentrated on the single

	Evidence in the pilot	Status at this scale
H1	Each skill left out of training stayed at chance in every seed; without FOLLOW, COMPOSE was solved in 0 of 4 runs, against 6 of 8 with it (E2).	Supported, after narrowing
H2	Models that memorised their textbooks were at chance on new ones (E1). No direct test.	Not tested
H3	At a fixed number of episodes, the number of distinct textbooks decided between memorising and reading (E1). Rule families were not varied.	Supported in part
H4	With the same parameters, a second loop extended the chain length that was solved; a third made no visible difference. At equal depth, looped and unshared models showed no consistent difference (E3).	Supported in part
H5	Separately trained skills did not combine: MIXED was solved in 0 of 16 runs, and chains longer than those trained stayed at the guessing level (E2, E3).	Contradicted

Table 6: The pilot against the hypotheses. “At this scale” means models of one to three hundred thousand parameters, one synthetic domain, one to four seeds.

correct cell does, and from a diffuse start there is little gradient leading towards it. When rows are arbitrary maps, the symbols that occur more often in row j are more likely to be the answer to a question about f_j , so attending to the right row already lowers the loss, and from there the model can sharpen onto the right cell. If this account is correct, it is a statement about curriculum design: a reading skill may be learnable in practice only when the training distribution also rewards a cruder version of it.

6.5 What did not work

Retrieval by content was not learned within our budget. In the shuffled layout a value must be found by matching the function name and the key against free-standing facts. In exploratory runs (Appendix C; single seeds, not controlled comparisons), FOLLOW stayed at chance in that layout with permutations and with arbitrary maps, for as long as we trained (between 1,500 and 4,000 steps, depending on the run). The aligned layout replaces retrieval by content with retrieval by address, and all results in this paper are for that easier setting. Retrieval by key is part of layer C1, so the pilot leaves the hardest primitive of the proposed curriculum untested.

6.6 Where the pilot leaves the hypotheses

Table 6 sets the pilot against the five hypotheses of Section 7. H1 to H4 were written down before any experiment was run. H1 is given in the form to which the pilot led us to narrow it: originally it predicted that removing rule following would lower accuracy on *every* exercise type, whereas INDUCE, which in TinyTextbook needs no table look-up, was still solved by 2 of the 4 seeds trained without FOLLOW. The prediction is now restricted to skills that build on the one removed. H5 was added during the pilot, after exploratory runs had already suggested its outcome, so it should be read as a question put to the full-scale study and not as a prediction that was tested fairly here.

7 Protocol for a Full-Scale Study

The pilot establishes the instruments. This section fixes, before the fact, what a study at practical scale should measure and what outcomes would count against the hypothesis.

Models. Decoder-only transformers at roughly 10M, 30M and 100M parameters, each in an unshared version and in a recurrent-depth version of matched parameter count, with at least five seeds per configuration.

Domain. TinyTextbook extended along four axes: (i) the shuffled layout, in which facts must be retrieved by content; (ii) more rule families, including operators given by tables on finite sets, finite algebraic structures, string-rewriting systems, and notations that are themselves defined in the material; (iii) larger alphabets and longer material, up to several thousand tokens; (iv) material rendered in controlled natural language as well as in symbols. Rule families are split into training families and held-out families so that G3 is measurable.

Curriculum conditions. A full model trained on C0–C5; six leave-one-layer-out models; and one model that receives, in addition to the full curriculum, a fixed corpus of declarative facts placed in the weights by ordinary training.

Hypotheses.

H1 (necessity is ordered). A layer left out of training is not acquired, and neither are the layers that build on it, even when those remain in training: removing C1 or C2 also removes C4, whereas removing C3 removes only what requires induction. *Counts against:* a leave-one-out model that matches the full model at G1 on the removed skill, or on a skill that builds on it.

H2 (facts do not help reading). Adding declarative facts to the weights does not raise accuracy at G1–G3 on textbooks that do not use those facts. *Counts against:* a consistent gain across seeds.

H3 (variety beats volume). At a fixed number of training episodes, the number of distinct textbooks, and of distinct rule families, determines accuracy at G1 and G3 more than the number of exercises per textbook. *Counts against:* G1 accuracy that depends on the episode count alone.

H4 (loops buy depth). At a fixed parameter count, more recurrent iterations raise the largest chain length that is solved at G1 without a written scratchpad. *Counts against:* no gain in solved chain length over an unshared model with the same parameters.

H5 (skills transfer upward). A model trained on the full curriculum solves G2 exercises that mix skills never mixed in training at a rate well above a model that lacks one of the mixed skills. *Counts against:* G2 accuracy at chance for the full model at every scale.

Analysis. All accuracies are reported at G0–G3 separately, per exercise type, as means over seeds with intervals, and as curves over training steps, because in-context learning can appear and later disappear during training (Singh et al., 2023). A condition is called a failure only relative to a stated chance level and a stated training budget.

Design corrections taken from the pilot. Three weaknesses of the pilot should not be repeated. (i) The training budget should be set so that the full-curriculum model converges in every seed, and the learning rate should not decay to zero at a fixed step, since late transitions were cut off by the schedule. (ii) When an exercise type is removed, the number of exercises of each remaining type should be held fixed, so that leaving

a skill out does not also raise the exposure of the others. (iii) Results should be reported as the fraction of seeds that solve a test as well as the mean, since outcomes are bimodal.

8 Discussion and Limitations

What the pilot suggests about the hypothesis. Three revisions to the proposal of Section 4 follow from the results.

First, *a base curriculum is ordered, not just a set*. Models trained on two-step exercises without one-step exercises acquired neither in any seed, although a two-step exercise contains all the supervision that a one-step exercise does. The lower layer was needed as a stepping stone to the higher one, in line with the old observation that it helps to start small (Bengio et al., 2009; Elman, 1993). The symmetry effect of Section 6.4 points the same way from another direction: whether a skill was learned within the budget depended on whether the training distribution rewarded a cruder version of it.

Second, *the curriculum has to list combinations as well as skills*. Models that could follow, compose and induce, each without error, could not apply an induced rule inside a composition. At this scale, having two skills in the weights did not provide their combination. Either the curriculum must train combinations explicitly, in which case the interesting question becomes how many combinations are enough for the rest to follow, or the ability to combine appears only with scale; the pilot cannot tell these apart. The same holds for chains longer than those seen in training, which no model followed.

Third, *variety is the resource to budget*. With the number of training episodes held fixed, the number of distinct textbooks decided whether the model read or memorised. For a curriculum designer this means that the cost of a skill should be counted in distinct rule sets, not in examples.

Limitations. The pilot was designed to be cheap, and its limits are severe.

- *Scale*. The models have between one and three hundred thousand parameters. Nothing here is evidence about models with a hundred million.
- *Retrieval by address, not by content*. All results use the aligned layout, in which the model knows where on the page each value sits. Retrieval by content was not learned within our budget (Section 6.5). What the pilot shows is that a small model can use a table whose layout it knows and whose contents it has never seen; that is weaker than reading a text.
- *One domain*. Five symbols, three tabulated functions, one family of induced rules, chains of at most six steps, no natural language.
- *Few seeds and a fixed budget*. Conditions have between one and four seeds. Learning in this setting proceeds in abrupt steps whose timing varies between seeds, and the learning rate decays to zero at a fixed step, so a run that scores at chance may have been a few hundred steps away from the transition. “Not learned” always means “not learned within the stated budget”.
- *Untested layers and levels*. C0 is assumed by the layout, C5 is not tested, and neither is G3.
- *No per-condition tuning*. Hyperparameters were chosen in exploratory runs with the full training mix and reused everywhere, which may favour some conditions.
- *A mild distribution shift at test time*. Training tables are arbitrary maps and test tables are permutations. This keeps the guessing level of a look-up exact and works against the models.

Latent computation and oversight. A model that composes rules across loops, without writing the intermediate values, performs reasoning that cannot be read from its output. In an open-book setting the

facts it relies on are at least visible in the context, but the steps are not. Because the legibility of intermediate reasoning is regarded as a safety-relevant property (Korbak et al., 2025), the full-scale study should include a condition in which intermediate values are written to a scratchpad (Nye et al., 2021) and should report what the unwritten version gains over it.

Relation to retrieval. Our question is independent of how the material reaches the context. A retrieval system decides which pages to show (Lewis et al., 2020); the base curriculum is about what the reader of those pages must already be able to do. If that turns out to be small, a small reader paired with a good library becomes an attractive design.

9 Conclusion

We asked how little a small model needs to know if the rest can be read, and proposed to answer it empirically: state a candidate base curriculum, build a benchmark in which memorising the rules is useless, remove one skill at a time, and measure what breaks at each level of generalisation.

The pilot shows that the programme can be run on a laptop-class CPU and that most of its answers are clear even through considerable seed-to-seed noise. At about a hundred thousand parameters, a transformer trained on enough distinct textbooks used textbooks it had never seen, and one trained on too few memorised them, with no visible difference on the training material. Each skill had to be trained to be present, the simplest skill had to be present for the next one to be learned, and two trained skills did not combine without being trained together. Applying the same layers twice bought computation depth without parameters.

The pilot also shows how far the programme is from its goal. The models read a page whose layout is fixed; retrieval by content, on which reading any real text depends, was not learned within our budget, parsing was built into the layout, and verification was not tested at all. Whether a small base curriculum exists for models of practical size therefore remains open. The protocol of Section 7 says what to measure to find out, and the code released with this paper is enough to start.

Acknowledgments and Disclosure of AI Assistance

This work made substantial use of a large language model (Claude, Anthropic). The research question and the central idea, that in-context learning could let small models trade stored knowledge for reading skill and that the open problem is to identify the minimal curriculum that must remain in the weights, are the author’s. The AI system was used to search and summarise the literature, to draft the text, to write the benchmark and training code, and to run the pilot experiments. The title, authors and year of every reference that has an arXiv identifier were checked against the arXiv metadata record, and the results in the tables and figures, together with the numbers quoted from them in the text, are generated directly from the run logs that are released with the paper. The author has reviewed the manuscript and takes full responsibility for its contents.

References

- Ekin Akyürek, Dale Schuurmans, Jacob Andreas, Tengyu Ma, and Denny Zhou. What learning algorithm is in-context learning? Investigations with linear models. In *International Conference on Learning Representations*, 2023. arXiv:2211.15661.
- Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyouon Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. Mixture-of-recursions: Learning dynamic recursive depths for adaptive token-level computation. *arXiv preprint arXiv:2507.10524*, 2025.

- Yu Bai, Fan Chen, Huan Wang, Caiming Xiong, and Song Mei. Transformers as statisticians: Provable in-context learning with in-context algorithm selection. *arXiv preprint arXiv:2306.04637*, 2023.
- Yoshua Bengio, Jérôme Louradour, Ronan Collobert, and Jason Weston. Curriculum learning. In *International Conference on Machine Learning*, 2009.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, 2020. arXiv:2005.14165.
- Stephanie C. Y. Chan, Adam Santoro, Andrew K. Lampinen, Jane X. Wang, Aaditya Singh, Pierre H. Richemond, Jay McClelland, and Felix Hill. Data distributional properties drive emergent in-context learning in transformers. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2205.05055.
- Yanda Chen, Ruiqi Zhong, Sheng Zha, George Karypis, and He He. Meta-learning via language model in-context tuning. In *Annual Meeting of the Association for Computational Linguistics*, 2022. arXiv:2110.07814.
- Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal transformers. In *International Conference on Learning Representations*, 2019. arXiv:1807.03819.
- Ronen Eldan and Yuanzhi Li. TinyStories: How small can language models be and still speak coherent English? *arXiv preprint arXiv:2305.07759*, 2023.
- Jeffrey L. Elman. Learning and development in neural networks: The importance of starting small. *Cognition*, 48(1):71–99, 1993.
- Xiang Fu. Can an easy-to-hard curriculum make reasoning emerge in small language models? Evidence from a four-stage curriculum on GPT-2. *arXiv preprint arXiv:2505.11643*, 2025.
- Shivam Garg, Dimitris Tsipras, Percy Liang, and Gregory Valiant. What can transformers learn in-context? A case study of simple function classes. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2208.01066.
- Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up test-time compute with latent reasoning: A recurrent depth approach. *arXiv preprint arXiv:2502.05171*, 2025.
- Angeliki Giannou, Shashank Rajput, Jy-yong Sohn, Kangwook Lee, Jason D. Lee, and Dimitris Papailiopoulos. Looped transformers as programmable computers. In *International Conference on Machine Learning*, 2023. arXiv:2301.13196.
- Chase Goddard, Lindsay M. Smith, Vudtiwat Ngampruetikorn, and David J. Schwab. When can in-context learning generalize out of task distribution? In *International Conference on Machine Learning*, volume 267 of *PMLR*, pages 19585–19599, 2025. arXiv:2506.05574.
- Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.
- Suriya Gunasekar, Yi Zhang, Jyoti Aneja, Caio César Teodoro Mendes, Allie Del Giorno, Sivakanth Gopi, Mojan Javaheripi, Piero Kauffmann, Gustavo de Rosa, Olli Saarikivi, et al. Textbooks are all you need. *arXiv preprint arXiv:2306.11644*, 2023.

- Shibo Hao, Sainbayar Sukhbaatar, DiJia Su, Xian Li, Zhiting Hu, Jason Weston, and Yuandong Tian. Training large language models to reason in a continuous latent space. In *Conference on Language Modeling*, 2025. arXiv:2412.06769.
- Tianyu He, Darshil Doshi, Aritra Das, and Andrey Gromov. Learning to grok: Emergence of in-context learning and skill composition in modular arithmetic tasks. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2406.02550.
- Noah Hollmann, Samuel Müller, Katharina Eggersperger, and Frank Hutter. TabPFN: A transformer that solves small tabular classification problems in a second. In *International Conference on Learning Representations*, 2023. arXiv:2207.01848.
- Alexia Jolicoeur-Martineau. Less is more: Recursive reasoning with tiny networks. *arXiv preprint arXiv:2510.04871*, 2025.
- Louis Kirsch, James Harrison, Jascha Sohl-Dickstein, and Luke Metz. General-purpose in-context learning by meta-learning transformers. *arXiv preprint arXiv:2212.04458*, 2022. NeurIPS 2022 Workshop on Meta-Learning.
- Tomek Korbak, Mikita Balesni, Elizabeth Barnes, Yoshua Bengio, Joe Benton, Joseph Bloom, Mark Chen, Alan Cooney, Allan Dafoe, Anca Dragan, et al. Chain of thought monitorability: A new and fragile opportunity for AI safety. *arXiv preprint arXiv:2507.11473*, 2025.
- Brenden M. Lake and Marco Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *International Conference on Machine Learning*, 2018. arXiv:1711.00350.
- Brenden M. Lake and Marco Baroni. Human-like systematic generalization through a meta-learning neural network. *Nature*, 623:115–121, 2023.
- Nayoung Lee, Kartik Sreenivasan, Jason D. Lee, Kangwook Lee, and Dimitris Papailiopoulos. Teaching arithmetic to small transformers. *arXiv preprint arXiv:2307.03381*, 2023.
- Mary I. Letey, Jacob A. Zavatone-Veth, Yue M. Lu, and Cengiz Pehlevan. Pretrain-test task alignment governs generalization in in-context learning. *arXiv preprint arXiv:2509.26551*, 2025.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020. arXiv:2005.11401.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172*, 2023.
- Sewon Min, Mike Lewis, Luke Zettlemoyer, and Hannaneh Hajishirzi. MetaICL: Learning to learn in context. In *Conference of the North American Chapter of the Association for Computational Linguistics*, 2022. arXiv:2110.15943.
- Samuel Müller, Noah Hollmann, Sebastian Pineda Arango, Josif Grabocka, and Frank Hutter. Transformers can do Bayesian inference. In *International Conference on Learning Representations*, 2022. arXiv:2112.10510.

- Alex Nguyen and Gautam Reddy. Differential learning kinetics govern the transition from memorization to generalization during in-context learning. In *International Conference on Learning Representations*, 2025. arXiv:2412.00104.
- Maxwell Nye, Anders Johan Andreassen, Guy Gur-Ari, Henryk Michalewski, Jacob Austin, David Bieber, David Dohan, Aitor Lewkowycz, Maarten Bosma, David Luan, et al. Show your work: Scratchpads for intermediate computation with language models. *arXiv preprint arXiv:2112.00114*, 2021.
- Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, et al. In-context learning and induction heads. *arXiv preprint arXiv:2209.11895*, 2022.
- Allan Raventós, Mansheej Paul, Feng Chen, and Surya Ganguli. Pretraining task diversity and the emergence of non-Bayesian in-context learning for regression. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2306.15063.
- Gautam Reddy. The mechanistic basis of data dependence and abrupt learning in an in-context classification task. *arXiv preprint arXiv:2312.03002*, 2023.
- Jacob Russin, Ellie Pavlick, and Michael J. Frank. The dynamic interplay between in-context and in-weight learning in humans and neural networks. *Proceedings of the National Academy of Sciences*, 122(35), 2025. arXiv:2402.08674.
- Nikunj Saunshi, Nishanth Dikkala, Zhiyuan Li, Sanjiv Kumar, and Sashank J. Reddi. Reasoning with latent thoughts: On the power of looped transformers. In *International Conference on Learning Representations*, 2025. arXiv:2502.17416.
- Aaditya K. Singh, Stephanie C. Y. Chan, Ted Moskovitz, Erin Grant, Andrew M. Saxe, and Felix Hill. The transient nature of emergent in-context learning in transformers. *arXiv preprint arXiv:2311.08360*, 2023.
- Johannes von Oswald, Eyvind Niklasson, Ettore Randazzo, João Sacramento, Alexander Mordvintsev, Andrey Zhmoginov, and Max Vladymyrov. Transformers learn in-context by gradient descent. In *International Conference on Machine Learning*, 2023. arXiv:2212.07677.
- Guan Wang, Jin Li, Yuhao Sun, Xing Chen, Changling Liu, Yue Wu, Meng Lu, Sen Song, and Yasin Abbasi Yadkori. Hierarchical reasoning model. *arXiv preprint arXiv:2506.21734*, 2025.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2201.11903.
- Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. An explanation of in-context learning as implicit Bayesian inference. In *International Conference on Learning Representations*, 2022. arXiv:2111.02080.
- Liu Yang, Kangwook Lee, Robert Nowak, and Dimitris Papailiopoulos. Looped transformers are better at learning learning algorithms. In *International Conference on Learning Representations*, 2024. arXiv:2311.12424.
- Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, et al. Scaling latent reasoning via looped language models. *arXiv preprint arXiv:2510.25741*, 2025a.

A Implementation Details

Tokens. The vocabulary has 17 tokens: padding, start of sequence, $\mathbb{Q}$, $=$, five symbols and eight function names. The aligned layout uses four of the names ($F1$, $F2$, $F3$ for the tabulated functions and one for the induced function). The material occupies 19 tokens after the start token: three rows of five values, then two key-value pairs for the induced function. An exercise with a chain of length d occupies $d + 4$ tokens including its answer.

Model. A decoder-only transformer with pre-layer normalisation, width 64, four attention heads, an MLP expansion factor of four with GELU activations, learned absolute position embeddings added to the token embeddings, rotary position encoding inside the attention, and an output layer that is not tied to the input embedding. A model described as “ n layers $\times$ r loops” applies the same stack of n layers r times in sequence, with no input re-injection and no loop-index embedding, followed by one final layer normalisation. The default model of experiments E1 and E2 is 2 layers $\times$ 2 loops (113,297 parameters).

Training. AdamW with learning rate 2×10^{-3} , $\beta = (0.9, 0.98)$, weight decay 0.01, linear warm-up over 200 steps followed by cosine decay to zero, gradient clipping at norm 1.0, and batches of 32 episodes with eight exercises each. E1, E2 and E4 train for 5,000 steps (160,000 episodes); E3 trains for 8,000 steps (256,000 episodes). The loss is the cross-entropy on answer tokens only. In training, each of the three tabulated functions is an independent uniformly random map on the five symbols (except in one arm of E4, where it is a uniformly random permutation); the induced function is a cyclic shift by a uniformly random non-zero amount, shown through two examples with distinct random keys.

Evaluation. Each test consists of 4,000 episodes with a single exercise each, generated from a random stream that is independent of the training stream. In tests on new textbooks the tabulated functions are uniformly random permutations. In the G0 tests of E1 the textbooks are drawn from the finite training pool and are therefore arbitrary maps. Curves during training use 500 episodes per test every 500 steps.

Guessing levels. On permutation textbooks the answer to a single look-up is uniform, so the guessing level of FOLLOW is 20%. In a chain, the same function can occur more than once, and the answer then equals the input x more often than $1/K$; for example $f(f(x)) = x$ whenever x lies on a cycle of length one or two, which has probability $2/K$. The best guesser that sees the exercise but not the tables answers x for the chains where this is the likeliest value and anything else otherwise. Enumerating all 120^3 triples of permutations and all chains gives its accuracy exactly: 26.7, 22.2, 23.7, 20.9 and 22.4% for chain lengths 2 to 6 (script `guess_levels.py`). For INDUCE the answer is never the input, so the guessing level is 25%. For MIXED at depth 2 the two functions are always different, and the level of 26.7% derived in Section 6 applies to a guesser that has induced g .

Compute. All 38 reported runs were executed on two virtual CPU cores without a GPU, using PyTorch 2.14 with one thread per run, and took 5.4 CPU-hours in total.

Release. The file `tinytextbook.py` contains the generator, the model, the training loop and the definitions of the four experiments; `make_results.py` regenerates every table, figure and number in this paper from the JSON logs. `guess_levels.py` computes the guessing levels. All three are available, together with the logs of every run, at <https://github.com/davitotty/Know-Less-Read-More>.

B Per-Run Results

Tables 7–10 list the final accuracy (%) of every run that enters the paper. “Min.” is the wall-clock training time in minutes.

Condition	Seed	Params	Steps	Min.	G0 fol.	G0 comp-2	G1 fol.	G1 comp-2
M=1	0	113,297	5,000	6.9	100.0	100.0	20.7	21.3
M=4	0	113,297	5,000	6.9	100.0	100.0	22.8	21.2
M=16	0	113,297	5,000	7.0	100.0	100.0	31.4	21.1
M=64	0	113,297	5,000	6.9	100.0	100.0	100.0	74.2
M=64	1	113,297	5,000	6.8	100.0	99.6	81.0	28.4
M= ∞	0	113,297	5,000	6.8	100.0	100.0	100.0	100.0
M= ∞	1	113,297	5,000	6.6	100.0	100.0	100.0	100.0

Table 7: E1, all runs.

Condition	Seed	Params	Steps	Min.	follow	comp-2	induce	mixed-2	comp-3
full	0	113,297	5,000	6.8	100.0	100.0	100.0	25.2	20.2
full	1	113,297	5,000	6.5	71.2	23.4	35.0	26.2	20.8
full	2	113,297	5,000	6.7	100.0	100.0	99.9	23.2	20.2
full	3	113,297	5,000	6.7	99.9	76.3	100.0	25.3	20.6
no_follow	0	113,297	5,000	7.0	19.1	26.4	100.0	25.1	20.5
no_follow	1	113,297	5,000	6.9	19.7	25.9	100.0	25.9	21.4
no_follow	2	113,297	5,000	6.7	19.4	25.0	91.8	17.4	21.7
no_follow	3	113,297	5,000	6.8	20.8	23.8	34.0	19.4	20.7
no_compose	0	113,297	5,000	6.2	100.0	17.4	100.0	16.7	22.3
no_compose	1	113,297	5,000	6.2	100.0	19.3	100.0	17.8	20.7
no_compose	2	113,297	5,000	6.2	20.4	20.6	99.3	21.8	19.7
no_compose	3	113,297	5,000	6.1	74.8	20.8	100.0	23.8	19.4
no_induce	0	113,297	5,000	7.2	100.0	100.0	20.7	9.1	19.8
no_induce	1	113,297	5,000	6.8	100.0	100.0	20.6	19.0	20.5
no_induce	2	113,297	5,000	7.0	100.0	100.0	20.6	20.0	20.3
no_induce	3	113,297	5,000	6.9	100.0	100.0	19.5	20.1	21.1

Table 8: E2, all runs.

Condition	Seed	Params	Steps	Min.	len 1	len 2	len 3	len 4	len 5	len 6
tied_2x1	0	113,297	8,000	7.1	100.0	24.9	24.4	22.9	21.4	22.1
tied_2x2	0	113,297	8,000	13.6	100.0	100.0	85.6	22.0	21.5	22.6
tied_2x2	1	113,297	8,000	13.7	100.0	100.0	100.0	23.0	21.2	21.4
tied_2x2	2	113,297	8,000	13.2	100.0	100.0	30.0	25.1	23.8	21.6
tied_2x3	0	113,297	8,000	20.6	100.0	100.0	46.7	23.2	23.0	22.3
untied_4	0	213,265	8,000	13.7	100.0	76.4	30.1	22.7	20.6	21.2
untied_4	1	213,265	8,000	13.8	100.0	100.0	100.0	48.1	21.8	23.0
untied_4	2	213,265	8,000	13.4	100.0	91.5	34.9	23.1	22.4	21.2
untied_6	0	313,233	8,000	22.7	100.0	95.2	65.1	25.1	22.1	22.8

Table 9: E3, all runs.

Condition	Seed	Params	Steps	Min.	perm.	maps
train_maps	0	113,297	5,000	5.9	100.0	100.0
train_maps	1	113,297	5,000	5.8	100.0	100.0
train_maps	2	113,297	5,000	5.7	100.0	100.0
train_perms	0	113,297	5,000	5.7	30.3	27.0
train_perms	1	113,297	5,000	5.7	52.5	42.1
train_perms	2	113,297	5,000	5.7	33.3	25.8

Table 10: E4, all runs. “perm.” and “maps” are FOLLOW accuracy on new textbooks that tabulate permutations and arbitrary maps.

C Exploratory Runs

The observations cited in Sections 6.4 and 6.5 come from runs made while the benchmark was being designed. Their logs were not retained, with the exception of items 6 to 8; experiment E4 repeats the comparison of items 3 and 4 under controlled conditions. They used a single seed each, were stopped early once the outcome seemed clear, and differ from one another in more than one factor, so they are reported as observations and not as controlled comparisons. Unless noted, the models had four unshared layers of width 64 and all other settings were as in Appendix A. Accuracies are on new permutation textbooks, where the guessing level of FOLLOW is 20%.

1. **Shuffled layout, permutations in training.** With free-standing `name key value` triples in random order, FOLLOW stayed at chance for as long as we trained: 2,000 steps with exercises in postfix or prefix order and learned absolute positions; 1,500 steps with rotary encoding only; 3,000 steps when training on FOLLOW alone with eight exercises per episode (one run at learning rate 10^{-3} with smaller textbooks mixed in, one at 3×10^{-3} without).
2. **One tabulated function.** Reducing the textbook to a single tabulated function plus the two examples of the induced one, with a two-layer model, left the training loss at $\ln 5$ for as long as we trained (3,000 to 3,800 steps), with and without an added next-token loss on all positions. A model of the same size fitted one fixed batch of 32 full-size episodes to a loss below 0.01 in 200 steps, so the failure is one of finding the general solution and not of capacity or of a faulty pipeline.
3. **Aligned layout, permutations in training.** FOLLOW was at chance at every checkpoint up to step 3,500 of a 4,000-step schedule at learning rate 10^{-3} , while INDUCE, whose two examples sit at fixed positions, reached 100% by step 1,500.
4. **Aligned layout, arbitrary maps in training.** With no other change, FOLLOW went from 32% at step 2,000 to 99.8% at step 2,500; at learning rate 2×10^{-3} it reached 98% by step 1,500.
5. **Shuffled layout, arbitrary maps in training.** FOLLOW was still at chance when the run was stopped at step 2,500.
6. **Four unshared layers on the E2 training mix.** With FOLLOW, INDUCE and COMPOSE at depths 2 and 3 in the training mix and learning rate 2×10^{-3} , FOLLOW and INDUCE reached 100%, but COMPOSE at depth 2 had only reached 29%, barely above its guessing level, when the run was stopped at step 7,000 of 10,000. This observation motivated both the looped default model and experiment E3; E3 then showed that the difference between looped and unshared models is not consistent across seeds, so the observation should not be read as evidence about architecture.
7. **Full curriculum with a 6,000-step schedule.** Before the budget was fixed, the full curriculum was run once with the final default model and a schedule of 6,000 steps (seed 0). It ended with FOLLOW and INDUCE at 100%, COMPOSE at depth 2 at 82.7% and MIXED at 21.7%. The run is not part of Table 3 because its budget differs from that of the reported runs; it is one more instance of a full-curriculum run that did not solve COMPOSE.
8. **Three loops with a 10,000-step schedule.** The three-loop model of E3, trained with a 10,000-step schedule (seed 0), was at 95.2% at chain length 3 by step 6,500 and still at the guessing level at length 4 when it was stopped at step 7,500.